

EduGenie: Google Gemini-Powered Learning Assistant

Project Description:

EduGenie is an AI-powered learning assistant designed to enhance students' learning experience using **Google Gemini AI**. The platform provides personalized academic support by helping users understand complex concepts, generate concise summaries, create flashcards, conduct question-answer sessions, generate quizzes, design study roadmaps, and prepare study timetables. EduGenie offers an interactive and user-friendly web interface that enables students to learn efficiently and improve their academic performance. It combines modern web technologies with Google's Gemini API to deliver accurate, fast, and intelligent educational assistance.

Scenarios:

Scenario 1

A student is preparing for semester examinations and wants to understand a difficult programming topic. The student enters the topic into EduGenie, and the system provides a detailed explanation along with a concise summary to improve understanding.

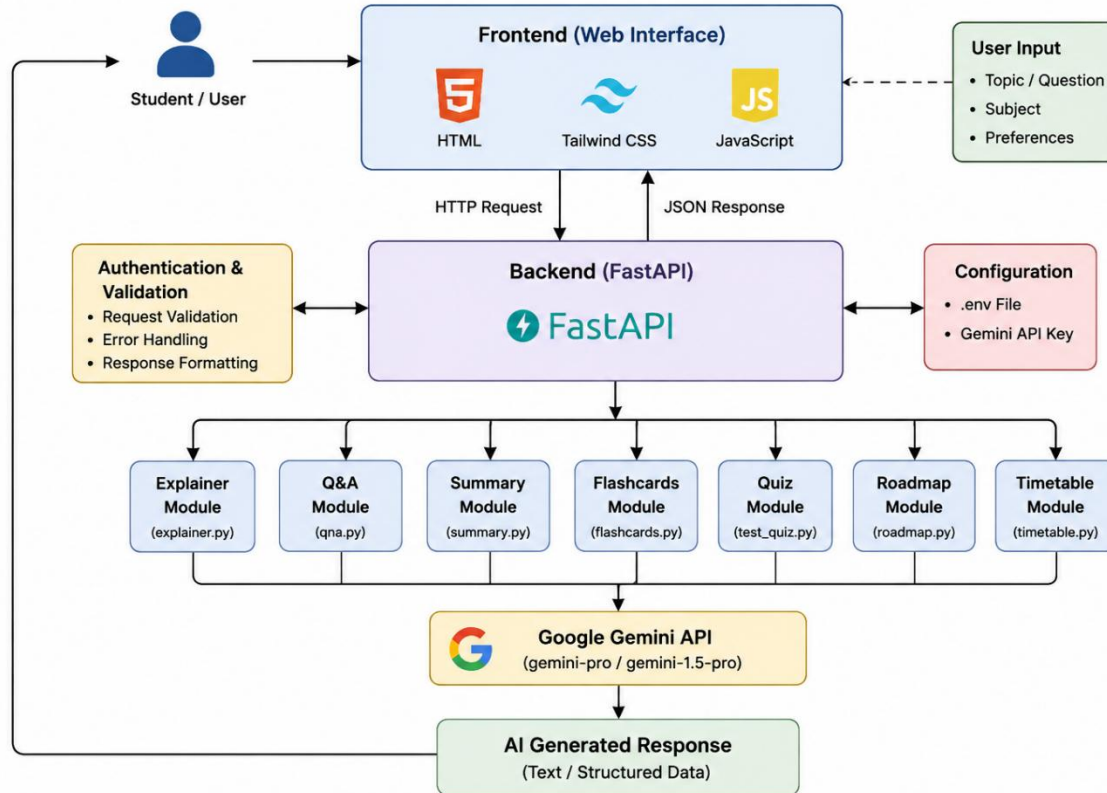
Scenario 2

A learner has completed a chapter and wants to revise quickly. EduGenie generates flashcards and a quiz based on the selected topic, allowing the student to test knowledge and strengthen memory.

Scenario 3

A student has limited time before examinations and needs a proper study schedule. EduGenie generates a personalized study roadmap and timetable based on the selected subjects, helping the student manage time effectively and prepare efficiently.

Technical Architecture:



Description:

EduGenie follows a modular client-server architecture to provide AI-powered educational assistance. The frontend, developed using HTML, JavaScript, and Tailwind CSS, offers an interactive user interface where students can submit learning queries. The FastAPI backend processes these requests and routes them to specialized modules such as Topic Explanation, Summary, Flashcards, Quiz, Roadmap, and Timetable. These modules communicate with the Google Gemini API to generate intelligent educational content. The generated responses are then returned to the frontend and displayed to the user in real time. This modular architecture improves scalability, maintainability, and overall system performance.

Pre-requisites:

- Python Programming Knowledge
- FastAPI Framework
- Google Gemini API
- HTML, CSS & JavaScript
- Git & GitHub
- Visual Studio Code
- Internet Connection

Project Workflow:

Milestone 1: Project Planning and Architecture

- Activity 1.1: Define the project objectives and identify the learning challenges faced by students.
- Activity 1.2: Select Google Gemini AI as the generative AI model for providing intelligent educational assistance.
- Activity 1.3: Design the overall system architecture, including the frontend, backend, AI modules, and API integration.
- Activity 1.4: Set up the development environment by installing Python, FastAPI, required libraries, and project dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop the core learning modules, including Topic Explanation, Question & Answer, Summary Generation, Flashcards, Quiz Generation, Study Roadmap, and Study Timetable.
- Activity 2.2: Implement the FastAPI backend to process user requests, validate inputs, manage routing, and communicate with the Google Gemini API.

Milestone 3: Backend Development

- Activity 3.1: Develop the main backend application (main.py) and integrate all AI modules into a unified API.
- Activity 3.2: Implement request validation, response processing, exception handling, and API response management to ensure reliable system performance.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop a responsive and user-friendly web interface using HTML, Tailwind CSS, and JavaScript.
- Activity 4.2: Integrate the frontend with the FastAPI backend to dynamically display AI-generated educational content.

Milestone 5: Testing and Deployment

- Activity 5.1: Test all modules, verify Google Gemini API integration, and resolve identified issues.
- Activity 5.2: Prepare the application for local deployment and ensure all required dependencies are properly configured.

Milestone 6: Documentation and Final Submission

- Prepare the complete project documentation, architecture diagrams, workflow diagrams, screenshots, and finalize the EduGenie project for submission.

Milestone 1: Project Planning and Architecture:

Activity 1.1: Generate Google Gemini API Key

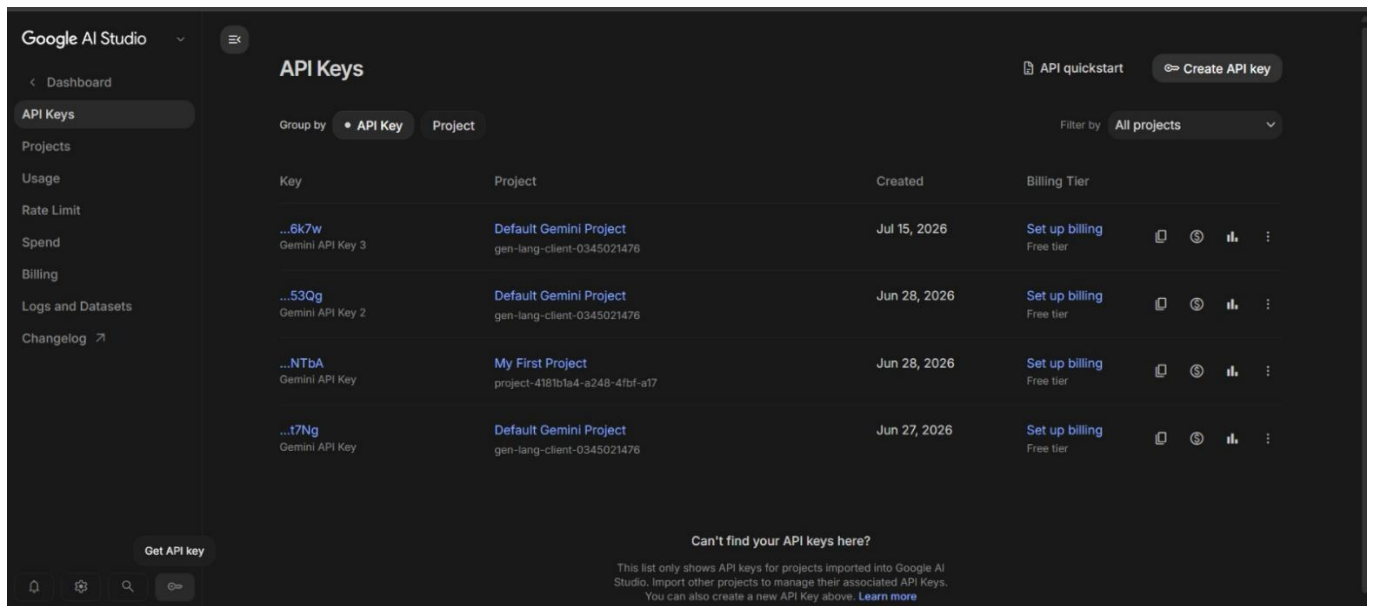
EduGenie uses the Google Gemini API to provide AI-powered educational assistance. The first step was to create a Google AI Studio account and generate an API key for secure communication with the Gemini model. The generated API key was integrated into the backend application and stored securely to authenticate all AI requests.

Steps:

- Step 1: Open Google AI Studio and sign in using a Google account.
- Step 2: Navigate to the API Keys section.
- Step 3: Click Create API Key.
- Step 4: Select the project and generate a new Gemini API key.
- Step 5: Copy the generated API key.
- Step 6: Store the API key securely inside the project configuration (.env) and use it in the backend for authentication.
- Step 7: Verify that the FastAPI application successfully connects to the Google Gemini API.

Screenshots:

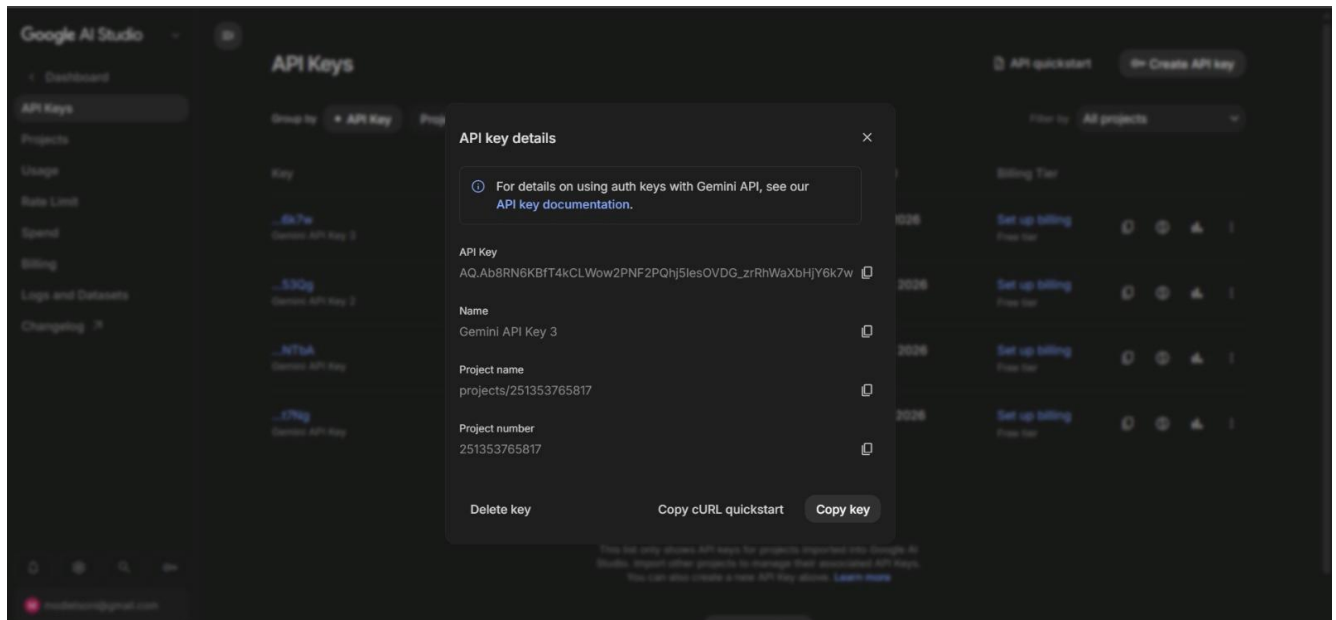
1. Google AI Studio – API Keys Dashboard:



The screenshot displays the Google AI Studio API Keys dashboard. On the left, there is a sidebar with navigation links: Dashboard, API Keys (selected), Projects, Usage, Rate Limit, Spend, Billing, Logs and Datasets, and Changelog. The main content area is titled 'API Keys' and includes a 'Group by' dropdown set to 'API Key' and a 'Filter by' dropdown set to 'All projects'. Below this is a table with the following columns: Key, Project, Created, and Billing Tier. The table contains four rows of API keys, each with a unique key ID, a project name, a creation date, and a billing tier of 'Free tier'. At the bottom of the dashboard, there is a 'Get API key' button and a message: 'Can't find your API keys here? This list only shows API keys for projects imported into Google AI Studio. Import other projects to manage their associated API Keys. You can also create a new API Key above. [Learn more](#)'.

Key	Project	Created	Billing Tier
...6k7w Gemini API Key 3	Default Gemini Project gen-lang-client-0345021476	Jul 15, 2026	Free tier
...53Qg Gemini API Key 2	Default Gemini Project gen-lang-client-0345021476	Jun 28, 2026	Free tier
...NTbA Gemini API Key	My First Project project-4187b1a4-a248-4f1bf-a17	Jun 28, 2026	Free tier
...t7Ng Gemini API Key	Default Gemini Project gen-lang-client-0345021476	Jun 27, 2026	Free tier

2. API Key Details:



Activity 1.2: Research and Select the Appropriate Generative AI Model

The following research activities were performed before selecting the most suitable Generative AI model for the EduGenie project:

- **Understand the Project Requirements:** Analyze the educational objectives of EduGenie, focusing on AI-powered features such as topic explanation, question answering, summary generation, flashcards, quizzes, study roadmaps, and timetable generation.
- **Explore Google Gemini Documentation:** Study the official Google Gemini API documentation to understand the available models, their capabilities, supported input formats, response quality, token limits, and API usage guidelines.
- **Evaluate Model Performance:** Compare different Gemini models based on response accuracy, reasoning capability, response speed, educational content generation, and suitability for student learning applications.
- **Select the Optimal Model:** Choose Google Gemini as the primary generative AI model because it provides accurate educational responses, strong reasoning ability, fast inference, and reliable support for multiple learning tasks such as explanations, summaries, quizzes, flashcards, and personalized study planning.

Activity 1.3: Define the System Architecture

The system architecture was designed to ensure modularity, scalability, and efficient communication between the frontend, backend, and Google Gemini AI services.

- **Design the System Architecture:** Create a modular client-server architecture consisting of a responsive frontend, a FastAPI backend, multiple AI processing modules, and Google Gemini API integration.
- **Define Frontend Responsibilities:** Design an interactive web interface using HTML, Tailwind CSS, and JavaScript where users can enter study topics or questions, select different learning features, and view AI-generated educational content.
- **Define Backend Responsibilities:** Implement the FastAPI backend to receive user requests, validate inputs, route requests to the appropriate AI module, process responses, handle exceptions, and communicate securely with the Google Gemini API.
- **Define AI Integration Workflow:** Configure the backend to generate structured prompts, send requests to the Google Gemini API, receive AI-generated responses, process the returned content, and display explanations, summaries, flashcards, quizzes, study roadmaps, and timetables through the frontend interface.

Activity 1.4: Set Up the Development Environment

The development environment for EduGenie was prepared by installing Python, FastAPI, and all required project dependencies. Visual Studio Code was used as the primary development environment, while Git was used for version control. After installing the required libraries from the requirements.txt file, the Google Gemini API key was configured, and the application was tested locally to ensure proper functionality.

Steps:

Step 1 – Install Python and Pip

Install **Python 3.10 or later** along with **pip** for dependency management.

Step 2 – Create a Virtual Environment

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

Step 3 – Install Project Dependencies

```
pip install -r requirements.txt
```

Step 4 – Configure the Google Gemini API Key

Create a .env file in the project root and add:

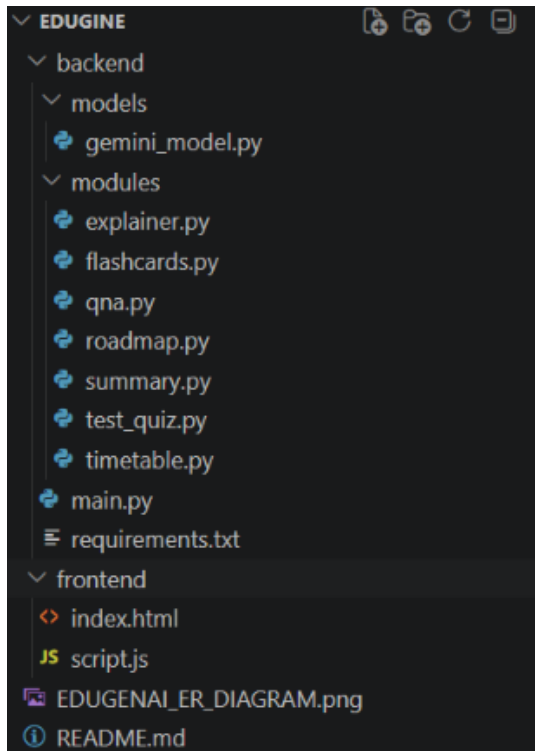
GEMINI_API_KEY=your_api_key_here

Step 5 – Run the Application

```
python main.py
```

Step 6 – Verify the Application

Open the application in your browser and verify that all modules (Topic Explanation, Summary, Flashcards, Quiz, Roadmap, and Timetable) are working correctly.



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core AI Learning Features

Implement the core AI-powered educational modules using **Google Gemini AI** to provide personalized learning assistance through a unified backend architecture.

- **Topic Explanation:** Generates detailed and easy-to-understand explanations for complex academic concepts.
- **Question & Answer:** Provides accurate answers to user queries with conceptual clarity.
- **Summary Generation:** Produces concise summaries of lengthy topics for quick revision.
- **Flashcard Generator:** Creates flashcards containing important questions and key concepts for effective revision.
- **Quiz Generation:** Generates topic-based quizzes to assess the learner's understanding and knowledge.
- **Study Roadmap:** Creates a structured learning roadmap to guide students through a subject systematically.
- **Study Timetable:** Generates personalized study schedules based on the user's learning requirements.

Activity 2.2: Implement the FastAPI Backend

Define API Endpoints

- Develop API endpoints in **FastAPI (main.py)** to process user requests and return AI-generated educational content.
- Organize the backend using separate modules such as **explainer.py, qna.py, summary.py, flashcards.py, test_quiz.py, roadmap.py, and timetable.py**.

Process User Input

- Create a user interface in **index.html** where students can enter study topics or academic queries.
- Send user requests asynchronously from the frontend to the FastAPI backend using JavaScript.
- Validate user input before processing to ensure meaningful and valid requests.

Integrate Google Gemini API

- Route incoming requests to the appropriate AI module based on the selected learning feature.
- Generate optimized prompts for Google Gemini AI to produce educational content.
- Process and validate AI-generated responses before sending them back to the frontend.
- Display generated explanations, summaries, flashcards, quizzes, roadmaps, and study timetables dynamically to the user.

Milestone 3: Backend Development

Milestone 3 focuses on developing the backend of EduGenie, which serves as the central processing unit of the application. The backend is developed using FastAPI and follows a modular architecture where each educational feature is implemented as an independent Python module. The backend receives user requests, validates the input, invokes the appropriate AI module, communicates with the Google Gemini API, and returns structured responses to the frontend. This architecture improves maintainability, scalability, and code reusability.

Activity 3.1: Develop the Main Backend Application (main.py)

Configure the FastAPI Application

The backend application is initialized using FastAPI. CORS middleware is enabled to allow secure communication between the frontend and backend. A request model (QueryPayload) is defined using Pydantic for validating incoming user requests.

Define the Core API Endpoint

EduGenie uses a dynamic API endpoint that processes different learning features based on the requested mode.

```
@app.post("/{mode}")  
async def handle_request(mode: str, payload: QueryPayload):
```

The endpoint supports the following learning modules:

- Question & Answer
- Topic Explanation
- Summary Generation
- Study Roadmap
- Quiz Generation
- Flashcards
- Study Timetable

The backend maps each mode to its corresponding Python module and executes the appropriate function dynamically.

Process User Requests

The FastAPI backend performs the following operations:

- Accepts user queries from the frontend.
- Validates the request using the QueryPayload model.

- Identifies the requested learning feature.
- Routes the request to the corresponding AI module.
- Returns the generated educational content as a JSON response.

AI Module Integration

EduGenie follows a modular backend architecture where every learning feature is implemented independently.

Integrated modules include:

- qna.py
- explainer.py
- summary.py
- roadmap.py
- test_quiz.py
- flashcards.py
- timetable.py

Each module creates a specialized prompt and sends it to the centralized Gemini integration module for response generation.

Activity 3.2: Integrate Google Gemini AI

Centralized AI Integration

All educational modules communicate with a centralized Gemini integration layer (gemini_model.py). This module securely loads the API key, configures the Google Gemini model, and provides a reusable function for generating AI responses. The application uses the Gemini 2.5 Flash model for fast and accurate educational content generation.

Prompt Engineering

Each module generates a customized prompt according to its learning objective.

Examples include:

- Question Answering
- Topic Explanation using the Feynman Technique
- Executive Study Summaries
- Flashcard Generation
- Quiz Generation
- Personalized Learning Roadmaps
- Weekly Study Timetables

These prompts ensure that the AI returns structured, readable, and educationally useful responses.

Response Processing

After receiving the AI-generated response, the backend:

- Processes the returned content.
- Formats the output into structured JSON.
- Sends the response back to the frontend for display.

JSON-based outputs are generated for Flashcards and Quiz modules to simplify frontend rendering.

Error Handling

The backend includes exception handling to improve reliability.

It handles:

- Invalid learning modes.
- Gemini API errors.
- Runtime exceptions.
- HTTP error responses.

Meaningful error messages are returned whenever an unexpected failure occurs.

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing an interactive, responsive, and user-friendly interface for EduGenie. The frontend is developed using HTML, Tailwind CSS, and JavaScript to provide an AI-powered learning workspace where students can access different educational modules, submit learning queries, and view AI-generated responses dynamically.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main index.html file as the primary interface of EduGenie containing a sidebar, navigation header, input workspace, and AI output workspace.
- Design a module-based navigation menu allowing users to access Question & Answer, Concept Explainer, Text Summarizer, Flashcards, Roadmap Planner, Test Quiz, and Timetable Builder.
- Organize the webpage using semantic HTML elements to improve readability, accessibility, and maintainability.

Design a Responsive Layout Using Tailwind CSS

- Develop a responsive user interface using Tailwind CSS with support for desktop, tablet, and mobile devices.
- Implement a modern learning workspace with sidebar navigation, responsive content cards, and an organized output panel.
- Integrate Google Fonts, Font Awesome icons, and Dark/Light theme switching to improve the overall user experience.

Create Interactive User Components

- Design dynamic input forms for each educational module based on the selected learning feature.
- Allow users to enter study topics, concepts, questions, or learning goals depending on the selected module.
- Display AI-generated educational content inside an interactive workspace with a clean and organized layout.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle User Requests Asynchronously

- Capture user input using JavaScript event listeners.
- Send asynchronous POST requests to the FastAPI backend using the Fetch API.
- Display a loading animation while the backend generates AI responses.

Render AI Responses Dynamically

- Dynamically display AI-generated explanations, summaries, roadmaps, flashcards, quizzes, and study timetables inside the output workspace.
- Render Markdown responses into formatted educational content for better readability.
- Generate interactive Flashcards and Quiz interfaces based on structured JSON responses received from the backend.

Provide Interactive Learning Features

- Enable users to switch between different learning modules without reloading the webpage.
- Provide copy, download, and workspace management options for generated educational content.
- Support interactive quiz evaluation and flashcard navigation to improve the overall learning experience.

Maintain User Interface State

- Preserve the selected learning module while interacting with the application.
- Clear or update the workspace dynamically after each request.
- Provide meaningful error messages whenever the backend or AI service returns an unexpected response.

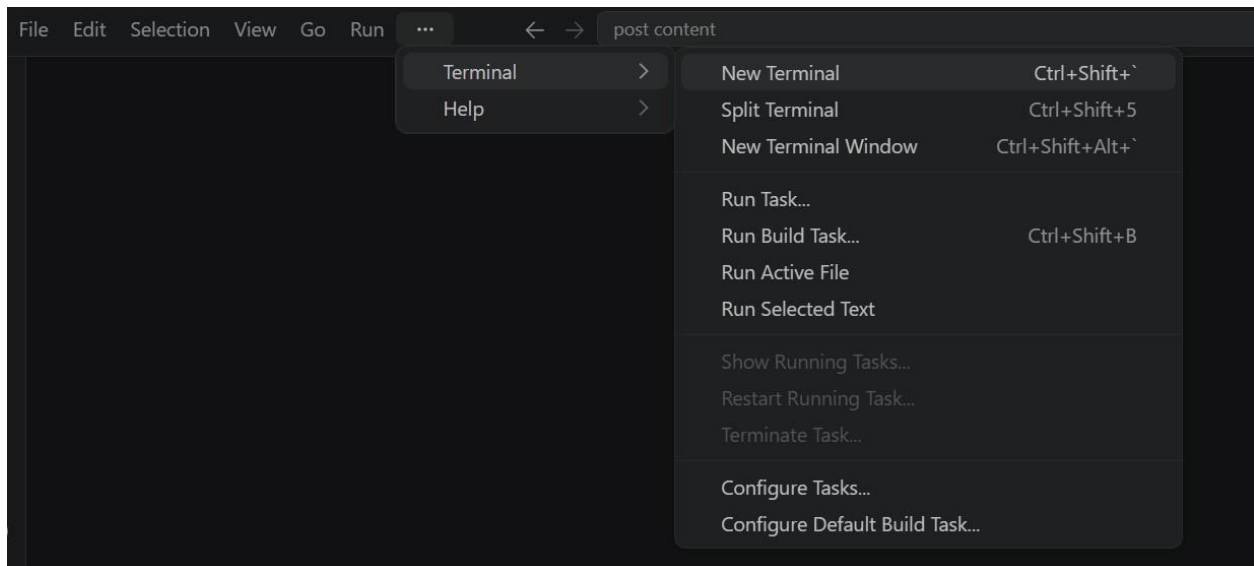
Milestone 5: Deployment

Milestone 5 focuses on deploying the EduGenie application and verifying that all AI-powered learning modules work correctly. The deployment process includes configuring the Python environment, installing project dependencies, running the FastAPI backend, testing the frontend locally, and optionally exposing the application publicly using Ngrok for demonstration purposes.

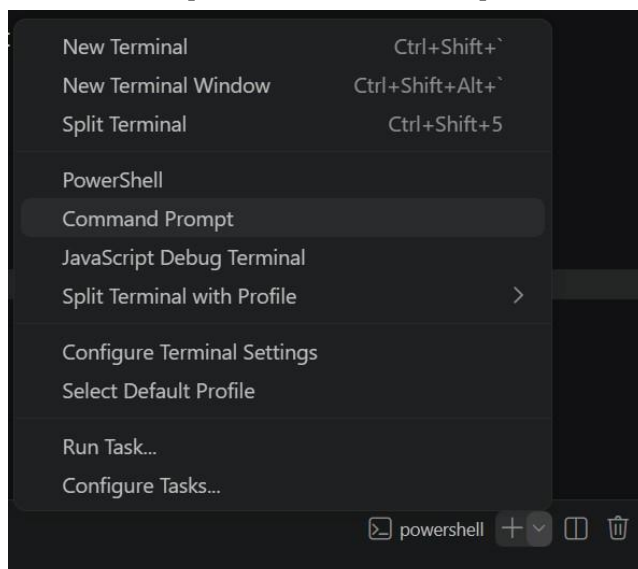
Activity 5.1: Preparing the Application for Local Deployment

Set Up The Development Environment:

- Open a New Terminal



- Then open "Command Prompt"



- (myenv) D:\projects>python app.py Create and activate a Python virtual environment.

```
D:\EduGenie> python -m venv venv  
D:\EduGenie> venv\Scripts\activate  
(venv) D:\EduGenie> pip install -r backend\requirements.txt
```

Configure Environment Variables:

- Create a .env file inside the backend directory.
- Add your **Google Gemini API Key** to the environment file.
GEMINI_API_KEY=your_google_gemini_api_key
- The application automatically loads the API key before communicating with the Google Gemini model.

Activity 5.2: Local Testing and Verification

Start the FastAPI Backend

- Run the FastAPI application locally using Uvicorn.

```
(venv) D:\EduGenie> uvicorn backend.main:app --reload
```

Launch the Frontend

- Open the frontend/index.html file in your web browser.
- Verify that the frontend successfully communicates with the FastAPI backend.

Verify Application Features

- Test all EduGenie learning modules, including:
 - Question & Answer
 - Concept Explainer
 - Text Summarizer
 - Flashcards
 - Test Quiz
 - Roadmap Planner
 - Timetable Builder
- Verify that AI-generated responses are displayed correctly.
- Test error handling by providing invalid or empty inputs.

Activity 5.3: Public Deployment using Ngrok

Ngrok creates a secure public URL that exposes the locally running FastAPI server over the internet. This allows the EduGenie application to be demonstrated without deploying it to a cloud hosting platform.

Install Ngrok

- Download and install the Ngrok client from the official website.
- Configure your Ngrok authentication token.

Start the FastAPI Server

Run the backend before starting Ngrok.

```
uvicorn backend.main:app --reload
```

Expose the Application Publicly

Open another terminal and execute:

```
ngrok http 8000
```

Ngrok generates a secure public URL similar to:

```
https://abcd-1234.ngrok-free.app
```

This public URL can be shared to demonstrate EduGenie without requiring local installation.

Verify Public Access

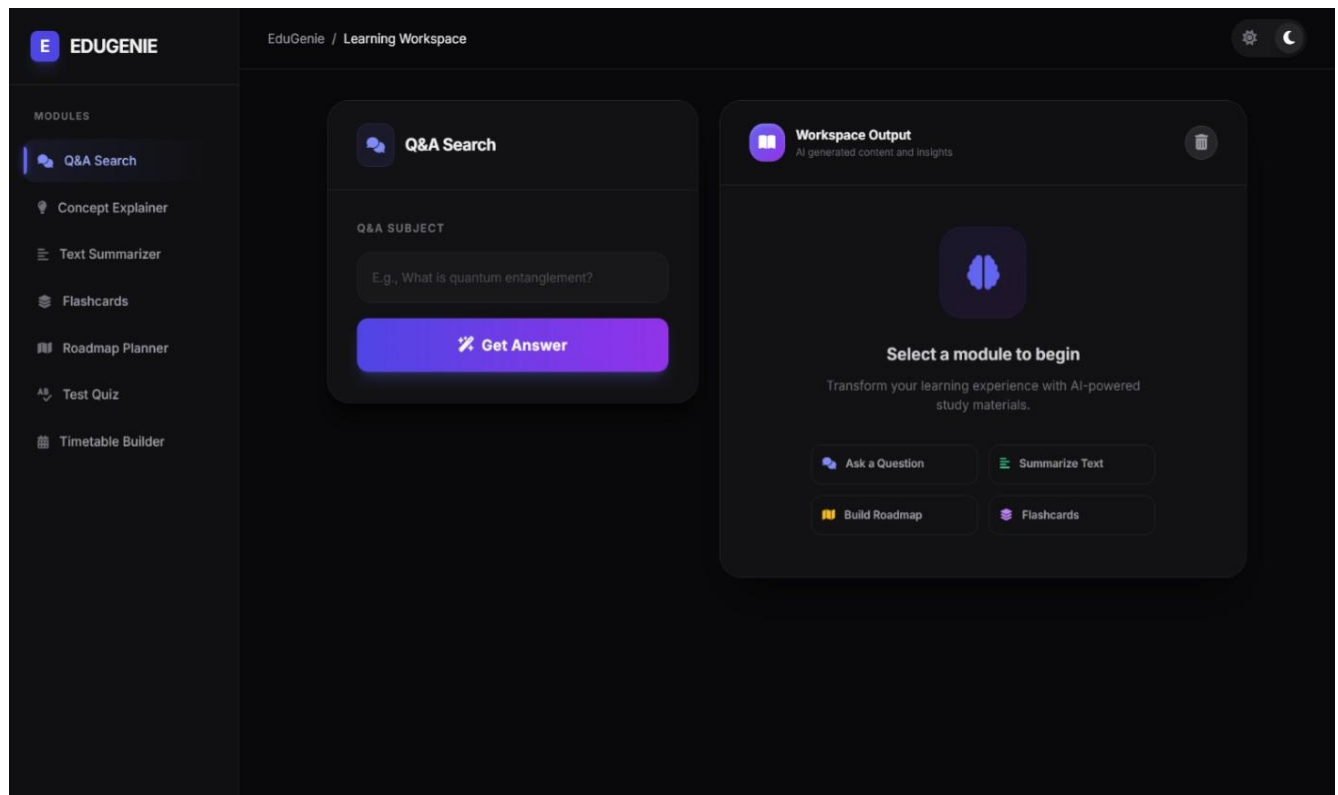
- Open the generated Ngrok URL in a web browser.
- Test all AI learning modules.
- Verify successful communication between the frontend, FastAPI backend, and Google Gemini API.

Important Notes:

- Ensure that the FastAPI backend is running before launching the frontend or starting Ngrok.
- Keep the Google Gemini API Key secure and never upload the .env file to GitHub.
- The free version of Ngrok generates a new public URL each time a new tunnel is started.
- An active internet connection is required because EduGenie communicates with the Google Gemini API to generate educational content.

Exploring the Website's Web Pages

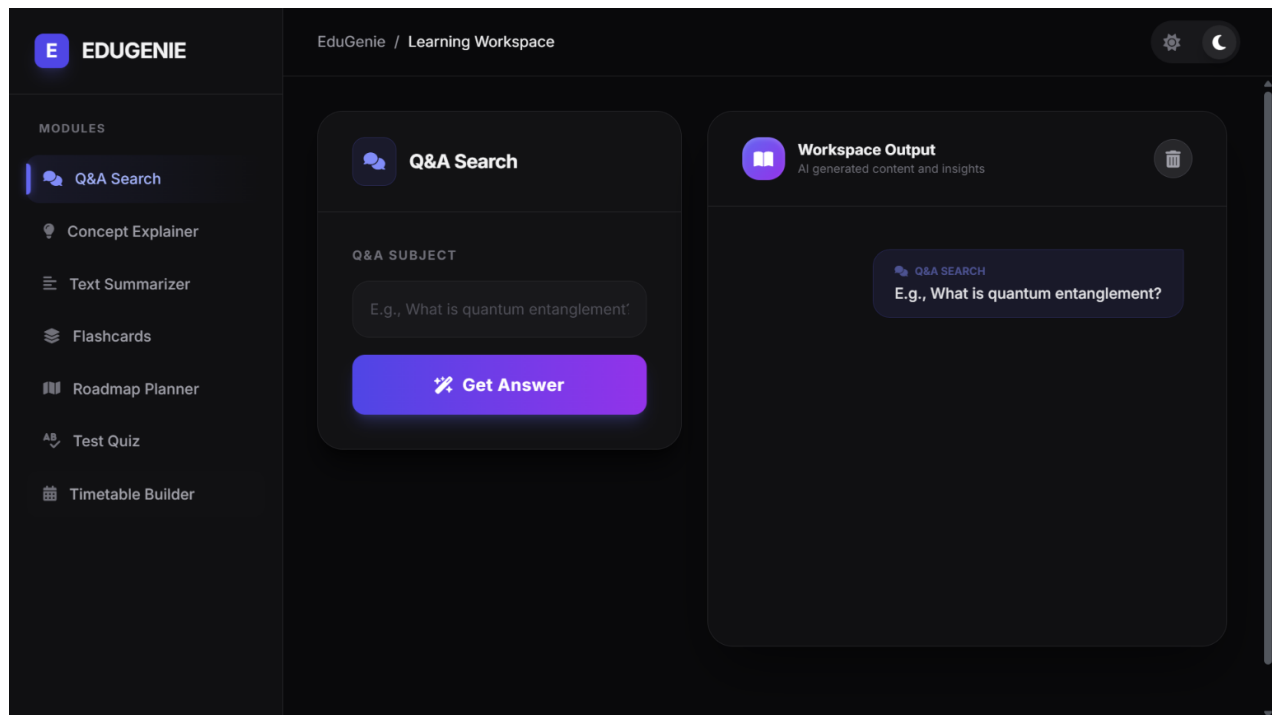
1. Home / Learning Workspace



Description:

The Home Page serves as the primary interface of EduGenie, providing an AI-powered learning workspace for students. It features a modern dark-themed interface with a navigation sidebar containing various learning modules, including Question & Answer, Concept Explainer, Text Summarizer, Flashcards, Roadmap Planner, Test Quiz, and Timetable Builder. The workspace is designed to provide an intuitive and user-friendly environment where students can easily access different AI-powered educational tools.

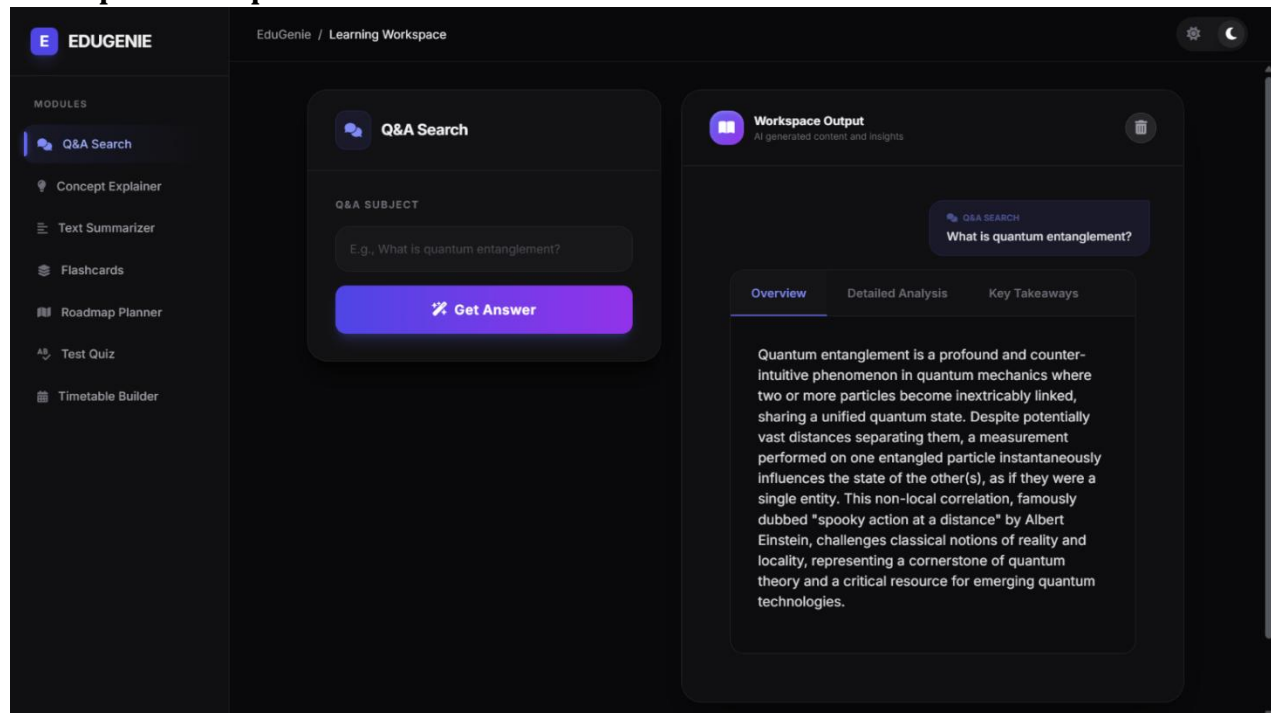
2. Input Workspace



Description:

The Input Workspace allows users to enter study topics, concepts, or academic questions based on the selected learning module. Users simply provide the required input and click the Get Answer button. The frontend validates the input and sends the request asynchronously to the FastAPI backend, which processes it using Google Gemini AI to generate intelligent educational content.

3. Output Workspace



Description:

The Output Workspace dynamically displays AI-generated educational content received from the Google Gemini API. Depending on the selected module, users can view detailed explanations, concise summaries, flashcards, quizzes, study roadmaps, or personalized timetables. The generated content is presented in a structured and easy-to-read format, enabling students to understand concepts effectively and improve their learning experience.

Conclusion:

EduGenie is an AI-powered educational platform developed using FastAPI, HTML, Tailwind CSS, and JavaScript, with Google Gemini AI as its core intelligence engine. The application provides an interactive learning environment where students can access multiple educational features, including Question & Answer, Topic Explanation, Text Summarization, Flashcards, Quiz Generation, Study Roadmaps, and Personalized Timetables. Through its modular backend architecture and responsive frontend design, EduGenie delivers fast, accurate, and personalized learning assistance while maintaining scalability, maintainability, and ease of use.

The project successfully covered all major phases of software development, including project planning, system architecture design, backend and frontend implementation, AI integration, testing, and deployment. By combining modern web technologies with Google Gemini AI, EduGenie demonstrates how generative AI can significantly improve students' learning experience by simplifying complex concepts, enhancing revision, and supporting effective study planning.

Looking ahead, EduGenie offers significant opportunities for future enhancement, including user authentication, cloud deployment, multilingual support, learning progress tracking, personalized recommendations, voice-based interaction, and mobile application development. With continuous improvements in AI capabilities and user experience, EduGenie has the potential to evolve into a comprehensive intelligent learning assistant that supports students across diverse educational domains.